

A Survey of Blockchain Data Management Systems

QIAN WEI, Shandong University, China

BINGZHE LI, Oklahoma State University, USA

WANLI CHANG, Hunan University, China

ZHIPING JIA and ZHAOYAN SHEN, Shandong University, China

ZILI SHAO, The Chinese University of Hong Kong, China

Blockchain has been widely deployed in various fields, such as finance, education, and public services. Blockchain has decentralized mechanisms with persistency and auditability and runs as an immutable distributed ledger, where transactions are jointly performed through cryptocurrency-based consensus algorithms by worldwide distributed nodes. There have been many survey papers reviewing the blockchain technologies from different perspectives, e.g., digital currencies, consensus algorithms, and smart contracts. However, none of them have focused on the blockchain data management systems. To fill in this gap, we have conducted a comprehensive survey on the data management systems, based on three typical types of blockchain, i.e., standard blockchain, hybrid blockchain, and DAG (**Directed Acyclic Graph**)-based blockchain. We categorize their data management mechanisms into three layers: blockchain architecture, blockchain data structure, and blockchain storage engine, where block architecture indicates how to record transactions on a distributed ledger, blockchain data structure refers to the internal structure of each block, and blockchain storage engine specifies the storage form of data on the blockchain system. For each layer, the works advancing the state-of-the-art are discussed together with technical challenges. Furthermore, we lay out several possible future research directions for the blockchain data management systems.

CCS Concepts: • **Information systems** → **Data management systems**; *Distributed storage*;

Additional Key Words and Phrases: Data management, blockchain architecture, blockchain data structure, blockchain storage engine

ACM Reference format:

Qian Wei, Bingzhe Li, Wanli Chang, Zhiping Jia, Zhaoyan Shen, and Zili Shao. 2022. A Survey of Blockchain Data Management Systems. *ACM Trans. Embedd. Comput. Syst.* 21, 3, Article 25 (May 2022), 28 pages. <https://doi.org/10.1145/3502741>

This work was supported by the National Key Research and Development Program of China under Grant no. 2019YFB2102600.

Authors' addresses: Q. Wei, Z. Jia, and Z. Shen (corresponding author), School of computer science and technology, Shandong University, Binhai road no.72, Jimo, Qingdao, China; emails: weiqian1212@mail.sdu.edu.cn, {jzp, shenzhaoyan}@sdu.edu.cn; B. Li, School of Electrical and Computer Engineering, Oklahoma State University, Stillwater, OK 74074, USA; email: bingzhe.li@okstate.edu; W. Chang, the College of Computer Science and Electronic Engineering, Hunan University, Changsha 410082, China; email: wanli.chang@york.ac.uk; Z. Shao, Department of Computer Science and Engineering, The Chinese University of Hong Kong, Room 909, Ho Sin-Hang Engineering Building, Shatin, NT, Hong Kong, China; email: zilishao@cuhk.edu.hk.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2022 Association for Computing Machinery.

1539-9087/2022/05-ART25 \$15.00

<https://doi.org/10.1145/3502741>

1 INTRODUCTION

Blockchain systems, such as Bitcoin [66], Ethereum [12], and HyperLedger [13], have received extensive attention in both academia and industries. A blockchain system can be regarded as a public ledger and transactions are committed to this ledger based on cryptographic primitives and distributed consensus mechanisms. Blockchain technology generally has key characteristics of decentralization, persistency, and auditability [107].

Currently, blockchain systems can be classified into three categories based on their chain structures: standard blockchain, hybrid blockchain, and **Directed Acyclic Graph (DAG)**-based blockchain. Standard blockchain systems, such as Bitcoin and Ethereum, store all committed transactions in a chain list of blocks. The chain grows by appending new blocks continuously. In a standard blockchain system, some allowable nodes can participate in the blockchain network as a node and maintain a full copy of the chain. For privacy protection, another architecture called hybrid blockchain, such as Hyperledger Fabric [13], and Ripple [6], is proposed to maintain multiple chains in the network, and each node is only allowed to access the data in specified chains. Different chains cooperate together to exchange information and issue cross-chain transactions. Different from standard blockchain and hybrid blockchain, DAG-based blockchain systems, such as Dag-Coin [48], IOTA [77], and Byteball [16], utilize DAG rather than block list as the basic data structure to manage transactions. In the DAG structure, each block is a transaction. When a transaction comes, it will connect two or more previous blocks in the DAG-based on node selection algorithms.

Though blockchain systems have shown great potential in various areas, they suffer from two critical issues: security and scalability. Blockchain systems promise their security based on cryptography and consensus algorithms. However, many of these security mechanisms have theoretical weaknesses that expose them to various attacks, such as Malware attacks, Eclipse attacks, and Time jacking. Scalability is another issue that seriously limits the deployment of blockchain systems from several perspectives as follows. First, a blockchain system usually has a very low throughput, ranging from a few to dozens of **transactions per second (TPS)**, which cannot deal with high-frequency trading scenarios. Second, with continuous operations of a blockchain system, the accumulative and excessive data load becomes a severe problem [40], which not only slows down the transaction verification process, but also requires longer network broadcasting time, thus, further limiting system throughput. Moreover, to participate in a transaction verification process, a node needs to synchronize all data of a blockchain system. As a result, this excessive data load of large-scale blockchain systems may hinder the participation of many resource-limited nodes. Third, the query process of a blockchain system is very time-consuming. To apply blockchain systems to applications in various fields [9], the query processing mechanism needs to be improved to provide low processing and response time with integrated query functions and high data security.

Some research studies investigate to improve the security and scalability of blockchain systems. For the security issue, many survey papers summarize the security problems and potential security threats in the current blockchain systems and introduce solutions to these security problems. For example, Lin and Liao [56] introduce several typical security issues in blockchain systems and summarize some of the current security challenges that need to be resolved. Li et al. [54] not only raise security issues and summarize solutions but also suggest some future directions in the security area. Anita and Vijayalakshmi [5] summarize vulnerable attacks and different approaches against these attacks for the blockchain systems with a taxonomy of the attacks and a review of the countermeasures. Liu and Liu [58] focus on dealing with the security assurance of blockchain smart contracts and the correctness verification of blockchain smart contracts. The scalability issue of blockchain systems includes low throughput, excessive data load, and inefficient query engines. All these issues are highly related to the data management

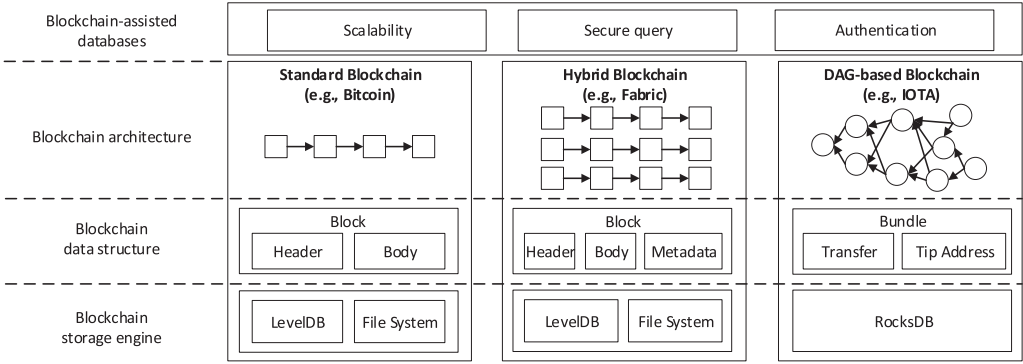


Fig. 1. The layers of blockchain data management stack.

system of blockchain. However, to the best of our knowledge, no survey papers provide a thorough investigation of blockchain data management.

In this article, we survey the technologies related to blockchain data management from three aspects, namely, blockchain architecture, blockchain data structure, and blockchain storage engine. Specifically, in the blockchain architecture, our focus is on the techniques for how to record transactions on a distributed ledger; for the blockchain data structure, we summarize the structures that are used to manage transactions on a chain; in the blockchain storage engine, our focus is on underlying data storage techniques for blockchain nodes. We introduce efforts that have been performed from these three aspects for three typical blockchain system designs: standard blockchain, hybrid blockchain, and DAG-based blockchain. Moreover, some potential research directions are discussed for blockchain data management.

The rest of the article is organized as follows. Section 2 introduces background information for standard blockchain, hybrid blockchain, and DAG-based blockchain. Sections 3, 4, and 5 survey the data management techniques for standard blockchain, hybrid blockchain, and DAG-based blockchain, respectively. Section 6 summarizes the database techniques for blockchain. Section 7 describes the related work and Section 8 concludes this article.

2 BACKGROUND

2.1 Blockchain Data Management Stack

As shown in Figure 1, the blockchain data management stack can be separated into four levels: the blockchain-assisted databases, blockchain architecture, blockchain data structure, and block storage engine. The blockchain architecture refers to the overall organizational structure of the blockchain network. Three typical blockchain designs (i.e., standard blockchain, hybrid blockchain, and DAG-based blockchain) use different architectures to maintain the distributed ledger. For instance, the standard blockchain adopts a list chain structure, the hybrid blockchain maintains several chains, and the DAG-based blockchain utilizes a DAG-based Tangle. The blockchain data structure refers to how the blockchain network organizes its transactions. The transactions of a blockchain system can either be organized into blocks by a mining process (**Proof-of-Work (PoW)** or **Proof-of-Stake (PoS)**) like in the standard and the hybrid blockchains or be directly attached to the network as a single node and verified by other nodes' confirmations like in the DAG-based blockchain. The blockchain storage engine is used to store and manage blockchain data on blockchain storage nodes. Blockchain systems usually adopt a database as the storage engine to maintain all data. Different blockchain systems may choose different database systems. Some blockchains choose a combination of a database and a file system to ensure data integrity [66].

Benefiting from the popular characteristics of the blockchain systems, people have deployed them as the assistant database to various scenarios, such as information sharing and private data protection. Depending on the different emphasis, we further divide the works which propose to adopt blockchain systems to assist the design of databases from three aspects: scalability, secure query, and authentication.

During the investigation process, we start from the four perspectives of the data management system, namely, blockchain architecture, blockchain data structure, blockchain storage engine, and blockchain-assisted databases. Based on these four aspects, according to the “standard blockchain,” “hybrid blockchain,” and “DAG-based blockchain” keyword search, we focus on the work done from the data management level and summarize the optimization work done for the data organization of blockchain systems.

2.2 Standard Blockchain

A standard blockchain system adopts a single chain list as its ledger. Transactions of the system are organized into a block and verified by a mining process. The verified blocks are added to the chain in chronological order. Blocks are linked according to their parent block hash in the block header until reaching the genesis block.

A transaction block is the basic data structure in a standard blockchain system and consists of a block header and block body. Block metadata is stored in the block header, and the transactions are stored in the block body. The contents of the block header of different blockchain systems are slightly different. These contents can be roughly divided into two sets. One set includes mining-related information such as timestamp, difficulty, and nonce, and the other set includes Merkle root, parent block hash, and block version.

The complexity of the data in the block header in standard blockchain systems is determined by its functional complexity. For example, the function of the Bitcoin system only includes the transfer of digital currencies, and thus the data in its blocks are relatively simple. Ethereum adds the concept of uncle blocks. Its block header also contains uncle hash. Therefore, the implementations of blockchain systems vary based on their different functions.

For the blockchain storage engine, a standard blockchain system usually adopts a file system to maintain its binary block data and a key-value database to maintain its metadata (e.g., Merkle tree and account information). For instance, Ethereum manages all its binary block data as normal files and adopts the key-value database LevelDB [33] to maintain its Merkle tree data to serve for account verification and query processes.

2.3 Hybrid Blockchain

Different from standard blockchain systems, hybrid blockchain systems are not public to all nodes but still offer blockchain features such as integrity, transparency, and security. The hybrid blockchain members can decide who can participate in the blockchain or which transactions are public. The blockchain architecture of a hybrid blockchain can have either one single listed chain as a standard blockchain system or several alliance chains. For the alliance chains, each chain corresponds to a set of ledgers, so each peer node in the blockchain may maintain multiple sets of ledgers. The structure of each chain is the same as a standard blockchain.

Similar to the standard blockchain system, the hybrid blockchain system also maintains transactions with the data structure of blocks. Each block mainly includes three parts: the relevant data of the block, the data related to its previous block, and the block metadata. The data of the block generally includes block hash and account number. Block metadata mainly includes the block creation time and signature.

For the blockchain storage engine, hybrid blockchain usually adopts a combination of databases and file systems. For instance, the storage engine of Hyperledger Fabric [13] is composed of LevelDB and a file system. The storage engine of Ripple [6] is composed of a relational database (SQLite) and a key-value store.

2.4 DAG-Based Blockchain

DAG-based blockchain system (e.g., IOTA) organizes all transactions in a DAG rather than packing transactions into blocks like in Bitcoin and Ethereum. Due to the non-block-based structure, DAG-based blockchains are not limited by block generation. Therefore, DAG-based blockchains have great potential to provide higher throughput and lower latency than block-based blockchains.

For the data structure, each transaction is represented by a block in the DAG-based blockchain system. The application logic of different DAG structures is not the same. For example, the DAG structure of IOTA is based on Tangle [77]. All the transactions are issued by clients through a selective PoW process [23, 78]. Leaf nodes (i.e., transactions that have not been approved by latter transactions) are called tips. For a transaction to be added, the system must select and approve two or more tips. During the tip selection and approval process, a random walking algorithm is used to choose tips for approval, and a transaction validation algorithm is performed to verify the transactions along the validation path.

For the blockchain storage engine, DAG-based blockchain usually adopts a relational or a key-value database to maintain its transactions and account information. For example, IOTA [77] adopts RocksDB to store all the transactions, the account information, and their corresponding addresses. Byteball [16] uses SQLite to store various types of data in different tables, such as the units table, to store the basic information of each unit.

In summary, standard blockchain is a kind of chained data structure that combines data blocks in chronological order. Each block composes multiple transactions and is generated by a mining process. Blocks are linked according to the parent block hash in the block header, all the way to the genesis block. Hybrid blockchain supports multiple alliance chains; each chain corresponds to a set of ledgers. So each peer node in the blockchain may maintain multiple sets of ledgers, and the structure of each chain is the same as the standard blockchain. DAG-based blockchain is based on the DAG structure. In the graph, each node is a transaction pointing to one direction and finally links to a root node. Standard blockchain is widely used in the fields of digital currency, financial asset transaction settlement, deposit certificates, and anti-counterfeiting data services. Hybrid blockchain is suitable for multi-organizational information exchange scenarios which can realize node transactions within and outside the organization. Since there are almost no transaction fees, DAG-based blockchain is more suitable for the Internet of Things environment.

For the blockchain scope, it is mainly divided into public, private, and consortium blockchains. The transaction records on the chain are visible to the public in the public blockchain, and all nodes can participate in the consensus. The private blockchain is restricted by centralized managers and can only be used by a small number of insiders. In the consortium blockchain, several organizations work together to maintain a chain, whose management is authorized. In the Standard blockchain and DAG-based blockchain, nodes can choose to join the public chain or build a private chain. Hybrid blockchain is a classic consortium blockchain, each consortium maintains its own ledgers, and nodes participating in the consortium can only view all information within the consortium chain in which it is located.

3 STANDARD BLOCKCHAIN

In this section, we first survey data management techniques for standard blockchain systems in terms of blockchain architecture, blockchain data structure, and blockchain storage engine.

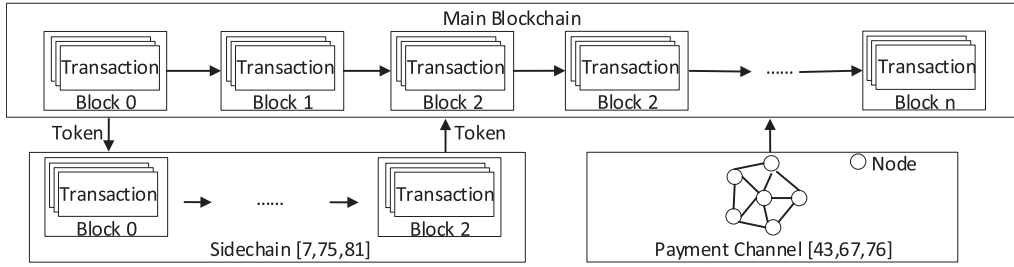


Fig. 2. Optimization for the standard blockchain architecture.

Table 1. Overview of Existing Optimization Papers for Standard Blockchain's Architecture

Target problem	Solution	Authors	Year	Weakness
The throughput bottleneck	sidechain	Back et al. [7]	2014	Lack of transaction details
		Poon and Buterin [75]	2017	Lack of transaction details
		Regnath and Steinhorst [81]	2018	Lack of transaction details
	micropayment channel	Poon and Dryja [76]	2016	Insufficient security
		Raiden Network [67]	2016	Insufficient security
		Kim et al. [43]	2021	Insufficient security

Then we list several future optimization directions for data management in standard blockchain systems.

3.1 Optimization for Standard Blockchain Architecture

The previous studies that optimize the standard blockchain architecture mainly aim at solving the throughput bottleneck. The essential idea of these works is to reduce the burden on the main network, such as offloading some complex transactions into off-chain execution. To achieve this, the techniques can be classified into two categories: sidechain [7, 75] and micropayment channel [67, 76]. Table 1 and Figure 2 summarize these optimization works for the standard blockchain architecture. As shown in Figure 2, the main idea of the sidechain is to build a small isolated chain with the same functionalities as the mainchain so that some transactions can first be carried out on the sidechain and then transferred to the mainchain. By doing so, some transactions are offloaded to the sidechains, thereby reducing the workload burden on the mainchain. In general, sidechain is a concept typically for interoperability. In another category, separate micropayment channels are adopted to conduct off-chain transactions between various nodes who participate in the blockchain network, and only the final results are recorded on the mainchain.

As a smaller isolated blockchain system, a sidechain is an independent chain with its own consensus mechanism and transaction type and runs its smart contract. Part of the transaction calculations and tasks of the mainchain are offloaded to the sidechain for processing. The basis for the sidechain is an **SPV-based (Simplified Payment Verification)** [7] decentralized two-way peg technology. In the SPV mode, a user will first issue a special transaction to transfer certain of its own tokens (digital assets) to a dedicated address on the mainchain. Then, this address will issue an SPV transaction to the sidechain to make these tokens circulate on the sidechain. The left tokens of the user on the sidechain can be organized into another transaction and be returned to the mainchain by sending the transaction to the dedicated address again.

Element chain [7] is a sidechain scheme to offload the heavy data workload for the mainchain of Bitcoin. Element chain achieves a two-way anchoring between itself and the Bitcoin mainchain. It not only inherits the token transaction function of the mainchain but also supports other

application scenarios by running smart contracts. Plasma [75] is a sidechain designed for the Ethereum mainchain. A sidechain root in Plasma is a smart contract running on the mainchain, which records the rules and the state hash of the sidechain. Multiple child chains can be generated from the root, which expands continuously and finally becomes a tree structure. Users can create a ledger on the Plasma chain and achieve asset-transfer between the Plasma chain and the Ethereum mainchain via the root. Regnath and Steinhurst [81] reduce verification steps by adding additional backlinks and enable embedded devices to verify blockchain content using only a few kilobytes of RAM. The conventional block structure is extended to only connect a block to its direct predecessor, by a leap-hash allowing the user to traverse the blockchain with a reduced amount of steps to verify the inclusion and integrity of block data. By doing so, transaction complexity decreases, and throughput increases, but at the same time it brings additional resource consumption.

Unlike a sidechain which can be regarded as the currency circulation between two independent blockchains, the micropayment channel is a transaction channel created between two nodes. In a decentralized system, transactions are sent over a network of micropayment channels (a.k.a. payment channels or transaction channels) whose transfers of value occur off-blockchain.

Two typical micropayment channel implementations are Lightning Network [76] and Raiden Network [67]. The Lightning Network [76] adds another layer to Bitcoin's blockchain, which enables users to create micropayment channels between any two parties. The bidirectional micropayment channel is contracts encumbered by hashlocks and timelocks. It is possible to clear transactions over a multi-hop payment network using a series of decrementing timelocks. Meanwhile, the Bitcoin transaction script in the channel is an implementation similar to "smart contract" to enable systems without trusted custodial clearinghouses or escrow services. These channels can exist as needed, and since they are established between two nodes, transactions are almost instant, and the fees will be extremely low or even zero.

Raiden Network [67], similar to Lightning Network, is the micropayment channel of Ethereum and is designed to be used with any **Ethereum Request for Comments 20 (ERC20)** compatible token. Raiden has four smart contracts, used to impose rule constraints on off-chain transactions. Currently, the routing in Raiden works in a simple manner. Unlike the Lightning Network that establishes a channel between two nodes, each node in the Raiden Network has a global view of the network. It knows the initial capacity of each channel by watching for the deposit events happening on the chain. Each node tries to forward the transfer through the shortest path with enough capacity to the target. If the transfer reaches the target and the protocol is followed properly, then all the pending transfers in the path will be unlocked and the node balances will be updated.

Another micropayment channel implementation aims at executing transactions in the form of smart contracts off-chain. Kim et al. [43] recorded the historical Ethereum state that was used when a transaction was originally executed. The transaction can then be replayed in isolation for testing purposes. In addition, the recorder reorganizes the Ethereum world state into transaction-relevant substates to execute smart contracts in parallel.

Although the above methods can offload transactions to sidechains or micropayment channels and improve the throughput of the mainchain for a blockchain system, off-chain transactions may face security issues, such as nodes cannot confirm the accuracy of transactions. Because transactions are performed off-chain without the consensus verification process of the blockchain system, some malicious transactions and tampering under the data chain may happen. At the same time, since only the results of these transactions are sent to the mainchain, the specific details of transactions are missed. Once an error occurs in the account balance in the latest state of the blockchain, historical transaction information needs to be traced back. Some off-chain transactions will not be recorded, and some nodes may be malicious or have suffered security attacks. In this case, it

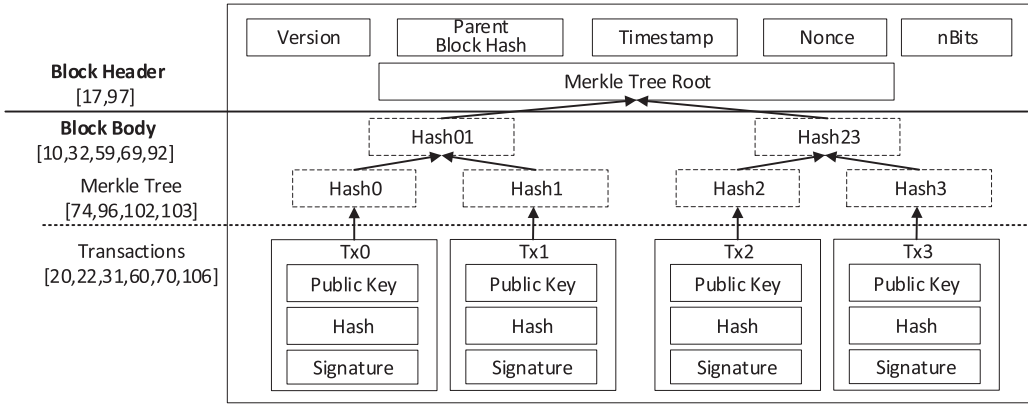


Fig. 3. Optimization for data structure of standard blockchain.

Table 2. A Brief Overview of Existing Optimization Papers of Standard Blockchain's Data Structure

Target problem	Solution	Authors	Year	Weakness
Excessive data load	Partial storage	Palai et al. [69]	2018	Only for transactions involving transferable entities
		Gao et al. [31]	2019	Cannot retain the historical traceability
		Bruce [10]	2014	Insufficient security
		Bitcoin Wiki. [92]	2019	Data loss
		Cross et al. [60]	2015	Additional storage load
	Block compression	Dai et al. [20]	2019	Increases the cost
		Xu [97]	2018	Data loss
		Bip152 [17]	2019	Increases the cost
		Ding et al. [22]	2019	Data loss
		Patsonakis and Roussopoulos [70]	2019	Total cost increase
Inefficient query performance	Redesigned ADS	Liu et al. [59]	2019	Data loss
		Ge et al. [32]	2020	Data loss
		Zheng et al. [106]	2018	Increases the cost
		Ponnapalli et al. [74]	2019	Decrease in decentralization
		Zhang et al. [102]	2021	Data loss

would be impossible to determine how many malicious transactions occurred among the results of an on-chain transaction.

3.2 Optimization for Standard Blockchain Data Structure

Table 2 and Figure 3 provide a summary of studies that optimize standard blockchain systems by redesigning their on-disk and in-memory data structures. Figure 3 shows the internal structure of a block, which mainly consists of a block header and block body. In this figure, we classify these studies into optimization for the block header [17, 97] and optimization for the block body [10, 69, 92]. For the optimization of the block body, most of the works [20, 22, 31, 60, 70] focus on improving the transaction part, and some [74, 96, 103] focus on improving the structure of the Merkle Tree.

3.2.1 Optimization for Excessive Data Load. To mitigate the excessive data load of standard blockchain systems, there are mainly three approaches, namely, partial storage, data compression, and simplified **authenticated data structure (ADS)**. The main idea of partial storage is to reduce the block size by removing a certain part of a block. In general, the removed part is the block body since some simple verification is done only depending on the block header, and most users are not

interested in the content of transactions issued by other users. For block compression, methods such as changing the smart contract form, encoding data, and so on, are proposed to compress the data in blocks. By doing so, the block size can be significantly reduced. ADSs such as Merkle trees are widely used in blockchain systems to verify if the data is correct and up-to-date. ADSs incur significant storage overhead due to random I/O, serialization, and hashing operations. Several studies propose to shard the tree and distribute the shards to different blockchain nodes [74].

Partial storage. For partial storage, Palai et al. [69] propose to perform the block summary of outdated blocks and only broadcast the block summary in blockchain systems. The block summary can store all the input resources for the given blocks and the total change introduced by these blocks. The proposed method allows resource-restricted light nodes to store an integrated blockchain so that it can validate transactions independently, which ultimately reduces dependency on full nodes. Though the work of verifying transactions only needs to include part of historical transactions, once there is a request to obtain information about other transactions, the node has to seek the assistance of other full nodes.

Gao et al. [31] propose to offload the block summary to an **InterPlanetary File System (IPFS)** and only maintain the updated transactions in the blockchain system. The scheme mainly achieves space optimization by summarizing blocks regularly, excluding the expired **Transaction Output (TXO)** that does not affect verification. The **Unspent Transaction Output (UTXO)** will be saved as a file on the IPFS network during optimization. In this way, the miner node can quickly get the file and start mining. The scheme summarizes the historical transaction data regularly, and eliminates the **Spent Transaction Output (STXO)** to effectively reduce the demand for node storage space.

Bruce [10] proposes a P2P crypto-currency scheme where old transactions can be removed from the network, thereby reducing the amount of storage. Since nodes only require the latest portion of the blockchain to sync with the network, they call this portion of the chain the “mini-blockchain.” Because the old transaction records are deleted, the account tree is used to fulfill the task of managing account balances and recording the ownership of coins. This also brings some potential benefits, such as faster transaction speed. On the other hand, deleting transaction records also brings potential security problems. Once some malicious transaction records are deleted, the account balance cannot be called back and corrected.

SPV [92] is proposed to only store the elementary information of each block, including hash, proof of work, and so on. No matter how large the future transaction volume is, the size of the block header always remains the same (80 bytes), which significantly saves storage space and reduces the burden on back-end users. Therefore, a user with limited-resource equipment is capable of loading a blockchain system under normal circumstances.

Segregated witness [60] is to remove the signature data in a transaction to reduce its size so that the block can accommodate more transactions, thereby reducing the overall data load. A new data structure called “witness” is defined to contain data required to check transaction validity but without data that determines transaction effects. In particular, scripts and signatures are moved into this new structure. At the same time, the witness also brings additional management and storage overhead.

Jigsaw-like Data Reduction (Jidar) [20] is a data reduction approach for the Bitcoin system. The main idea of Jidar is to allow nodes only to store transactions and relevant Merkle branches that they are interested in. Thus, it can alleviate the storage pressure of each node. The related Merkle branches are provided with a transaction to verify the validity of the newly proposed transaction. Given the requirement that a user wants to put all the fragments stored in different nodes into a complete block, just like stitching all the pieces into a complete jigsaw-puzzle picture,

a mechanism of querying full data is added to Jidar. This design neither has any trust assumptions for the Bitcoin system, nor sacrifices the original properties of the blockchain.

Block compression. Block compression reduces the data load by redesigning the block structure. Xu [97] presents section-blockchain, a new blockchain protocol, to solve the oversize storage problem without compromising the security of a blockchain. There is no full node and lightweight node. All nodes equally contribute to the section-blockchain network. The block storage mechanism is redesigned based on the original Nakamoto blockchain, which allows the nodes only keeping (1) the block headers in the mainchain; (2) a certain number of blockchain fragments; (3) a certain number of **Database Snapshots (DSs)**; and (4) a **Fragment and Database snapshot Grabbing Routing Table (FDGRT)**. Experiments demonstrate that section-blockchain is efficient, remarkably reduces the storage footprint, and withstands sudden data loss from massive nodes.

In the scenario where blockchain nodes are small-memory, resource-constrained devices, Ge et al. [32] redesign the blockchain constitution to mitigate the pressure of storage and calculation imposed on each node. Their solution relies on a voting mechanism combined with a reputation evaluation scheme. The item of reputation tree is added into the block and the root of the tree is recorded by the block header to further increase throughput, but the safety is relatively weaker than previous designs.

Liu et al. [59] present the concept of lightweight blockchain to solve the problem of excessive data load through two approaches. The first one is called **Unrelated Block Offloading Filter (UBOF)**. Through the analysis of **Unspent Transaction Output (UTXO)**, they propose the definition of **Unrelated Blocks (UBs)**. UBOF can detect and offload UBs, contributing to reducing the storage resources occupied by blockchain. The other is a lightweight data structure named **Light-Block (LB)**. They propose to broadcast LB instead of the entire block after one block is generated.

Compact block relay, or **BIP152 [17] (Bitcoin Improvement Proposal)**, reduces the amount of network requirement from P2P network nodes to broadcast blocks. The compact block relay changes the data structure of the original block in Bitcoin. As a result, a compact block only contains the header of the block and some short **transaction IDs (TXIDs)**, which are used to match the transactions already available to the receiver. The receiver node will use the received information and the transactions in its memory pool to reconstruct the entire block instead of transmitting the entire block. Txilm [22] is a protocol based on BIP152 that compresses transactions in each block to save the bandwidth of the network. In this protocol, a block carries the short hashes of TXIDs instead of completing transactions. Thus, the protocol achieves a higher block compression ratio. However, hash collisions more likely occur when a short hash is used. Therefore, Txilm optimizes the protocol using sorted transactions based on TXIDs to reduce the probability of hash collisions. Combined with the sorted transactions based on TXIDs, Txilm realizes 80 times of data size reduction compared with the original blockchains.

Patsonakis and Roussopoulos [70] introduce an alternative paradigm for developing smart contracts, in which the storage consumption is of constant size. This will significantly reduce the amount of storage consumed by smart contracts but cause a certain incentive imbalance due to the high cost. To solve this problem, they introduce recurring fees that are proportional to the state of smart contracts and adjustable by miners that maintain the network. At the same time, it also brings an increase in the total cost of the entire blockchain network.

Zheng et al. [106] propose an IPFS-based blockchain data storage model. With this model, miners deposit transaction data into the IPFS network and pack the returned IPFS hash of the transaction into a block. Utilizing the characteristics of the IPFS network and the features of the IPFS hash, the blockchain data can be significantly reduced.

Simplified ADS. Different from the methods mentioned earlier, Ponnappalli et al. [74] present a new ADS to reduce the excessive data load. A **Distributed Merkle Patricia Tree (DMPT)** is

designed to reduce the storage and computation overheads of Merkle trees and their variants. The DMPT eliminates storage overhead by storing the entire tree in memory. A DMPT vertically shards the Merkle Partica tree across the memory of multiple nodes, stored in the memory of a different node, eliminating the I/O bottleneck. Since both reads and writes in Merkle trees operate on a vertical path down the tree, vertical sharding enables process reads and writes on different nodes in parallel. DMPTs further help reduce network bandwidth utilization by combining novel techniques such as witness compaction and node bagging. Although sharding MPT on different nodes can greatly increase the degree of parallelism, it reduces the degree of decentralization since a single node in the network cannot independently complete verification and query work.

In summary, although the methods of partial storage and block compression significantly reduce the data load and increase system throughput, they will lead to a certain amount of information loss. For example, due to the imperfection of the data information, the query process becomes difficult. The node can only be used with the assistance of full nodes to complete the query on the specific transaction details, otherwise it can only verify whether the transaction is confirmed.

3.2.2 Optimization for Query Engine. Current blockchain systems lack a complete and efficient query system, the query efficiency is low, and the provided query interfaces are limited. The underlying data storage system of most standard blockchain systems uses **Log-structured Merge Tree (LSM-tree)**-based key-value stores, which sacrifice read performance in exchange for write performance. Moreover, the unstructured data storage system based on the key-value model does not support complex queries, which has become the main bottleneck to restrict query functions.

Xu et al. [96] propose a novel framework, called vChain, to alleviate the storage and computation cost of blockchain systems. The scheme employs verifiable queries to guarantee data integrity. To support verifiable Boolean range queries, they propose an accumulator-based authenticated data structure that enables dynamic aggregation over arbitrary query attributes. Two new types of indexes are developed to aggregate intra-block and inter-block data records for efficient query verification. They also propose an inverted prefix tree to accelerate the processing of a large number of subscription queries simultaneously. Although verifiable query ensures data integrity, it also brings high ADS maintenance costs.

By analyzing the performance of the existing techniques, Zhang et al. [103] propose a novel ADS, called GEM^2 -tree, which is not only gas-efficient but also effective in supporting authenticated queries. To further reduce the ADS maintenance cost without sacrificing query performance, they also propose an optimized structure, GEM^{2*} -tree, by designing a two-level index structure.

Zhang et al. [102] apply an authenticated query process, in which both the smart contract and the off-chain SP maintain an ADS named Merkle inverted index. The Merkle inverted index is an inverted index in which each keyword corresponds to a **Merkle B-tree (MBtree)** that indexes the corresponding object IDs. The smart contract maintains only the root digest of each MB-tree in the Merkle inverted index. This idea comes with the observation that for the on-chain ADS, only the root digests are used during the authenticated keyword search, which reduced the storage burden.

In summary, although these techniques improve the system query performance, they may induce a large overhead on the storage side due to storing the entire ledger. Hence, it becomes critical to handle excessive data load in blockchain systems. Particularly, even though these methods can improve the query efficiency, due to a large amount of data, the throughput of blockchain systems still remains low.

3.3 Optimization for Standard Blockchain Storage Engine

Table 3 and Figure 4 provide the summary of works that optimize blockchain data load, throughput, and query processing from the perspective of the blockchain storage engine. As shown in Figure 4,

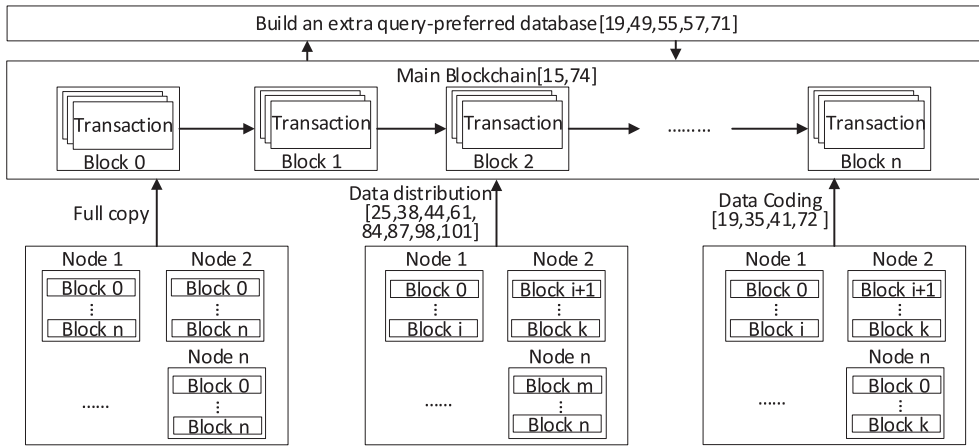


Fig. 4. Optimization for storage engine of standard blockchain.

Table 3. A Brief Overview of Existing Optimization Papers of Standard Blockchain's Storage Engine

Target problem	Solution	Authors	Year	Weakness
Excessive data load	Data distribution	Luu et al. [61]	2019	Decrease in decentralization
		Kokoris-Kogias et al. [44]	2019	Decrease in decentralization
		Zamani et al. [101]	2019	Decrease in decentralization
		Wang and Wang [87]	2019	Decrease in decentralization
		Han et al. [38]	2018	Decrease in decentralization
		Tao et al. [84]	2020	Increase in delay
	Data coding	Drakatos et al. [25]	2021	Increase the complexity
		Xu et al. [98]	2018	Decrease in decentralization
		Dai et al. [19]	2018	Increase query complexity
		Guo et al. [35]	2019	Increase query complexity
Inefficient query mechanism	Connect outside database	Jiang et al. [41]	2020	Increase query complexity
		Perard et al. [72]	2018	Increase query complexity
		Lesavich and Lesavich [49]	2017	Too much process cost
		Li et al. [55]	2017	Too much process cost
		Peng et al. [71]	2019	Too much process cost
Low throughput	Reduce I/O of tx	Linoy et al. [57]	2020	Too much process cost
		Hanley and Tewari [39]	2018	Excessive storage resource consumption
		Ponnappalli et al. [73]	2021	Increasing the complexity
		Chen et al. [15]	2021	Excessive storage resource consumption

the techniques for handling excessive data load are mainly categorized into distributed storage and data coding. To provide an efficient query engine, several studies propose to build an extra query-preferred database.

3.3.1 Optimization for Excessive Data Load.

Data distribution. The main idea of data distribution is that multiple nodes form a group with high trust to jointly process the arriving transactions. The current data distribution works are mainly divided into two levels of network and data. For the network level, most of the works use the idea of sharding [21] to distribute the transaction processing overhead among multiple smaller groups of nodes. Through network sharding, blockchain nodes are divided into different shards, and each shard forms an independent process and gets consensus on different transaction subsets. In this way, a subset of transactions that are not connected to each other can be processed in parallel, and transaction throughput can be significantly improved by orders of magnitude. As for the data level, the main idea is that multiple nodes form a group with high trust to jointly store all

data of a complete ledger. Therefore, each node is only responsible for hosting its own data, rather than storing the complete blockchain states.

For network sharding, Elastico [61] is the first sharding protocol for the permission-less blockchain. Technically, Elastico uniformly partitions or parallelizes the mining network into smaller committees, each of which processes a disjoint set of transactions (i.e., “shards”). Each committee has a reasonably small number of members so they can run a classical byzantine consensus protocol to decide their agreed set of transactions in parallel. After that, OmniLedger [44], a more recent distributed ledger based on the sharding technique, is proposed to solve the problems of Elastico. Specifically, to commit transactions atomically across shards, OmniLedger introduces an atomic commit protocol called Atomix. OmniLedger also adopts state blocks that are deployed along OmniLedger to minimize storage and update overhead. Rapidchain [101] reduces the amount of data exchange per transaction and does not need to gossip transactions to the entire network because of the usage of a fast cross-shard verification technique. Monoxide [87] is a scale-out blockchain that proposes Asynchronous Consensus Zones and scales the blockchain linearly while maintaining decentralization and security.

Tao et al. [84] implement a new distributed and dynamic sharding system. They mainly discuss a problem in sharding, which is that a small shard with few transactions tends to generate a large number of empty blocks resulting in a waste of mining power, while a large shard adversely affects parallel confirmations. An inter-shard merging algorithm with incentives to encourage small shards is proposed to merge and form a larger shard. On the other hand, different from the non-sharding systems, miners must communicate with each other to exchange their individual validation results to validate the transaction jointly.

As for data level, Xu et al. [98] introduce a novel concept called **Consensus Unit (CU)**, which organizes different nodes into one unit. The nodes in the same CU will work together and maintain at least an entire copy of blockchain data. Based on this idea, they further define a **Blocks Assignment Optimization (BAO)** problem to solve the optimal assignment of blocks such that the storage space is fully used and the query cost is minimized. Han et al. [38] develop a blockchain platform to support mobile devices. A novel concept called consensus unit which groups several single nodes in the network into a trusted cluster is introduced. The CU management module is used to monitor the status of the CU. The user can get the information of active peers and the ledger status of the current CU. When a user finds the ledger occupying too much storage space, it can raise optimizing storage requests with several sharding plans to CU peers.

Drakatos et al. [25] propose Triabase which divides the chain nodes into two types: nodes that store the entire shared database, and the others that use the database for their operations, such as sending query and update requests to the blockchain shared ledger. Triabase carries out machine learning at the edge, and abstracts machine learning in primitive blocks that are subsequently stored and retrieved from the blockchain.

Though the data distribution method efficiently mitigates the storage pressure, it weakens the decentralization of blockchain systems. The data distribution method requires a certain trust relationship among nodes, which is not required by the original blockchain design to ensure that data is not tampered with.

Data coding. Several studies propose to use erasure coding to process blockchain data and then perform distributed storage to mitigate the data load problem of a single node. Perard et al. [72] introduce a new type of node, an erasure-code-based low storage node, which stores coded fragments for each block of a blockchain. Similar to [72], Dai et al. [19] adopt a **network coded (NC) distributed storage (DS)** framework to store a blockchain to reduce storage requirements. NC applied to DS can easily rebuild the whole ledger from a small number of such nodes compared

with the method of simple replications. Correspondingly, when the system fetches these encoded blocks, it needs to perform decoding before the normal blockchain procedure.

Different from [72] and [19], Guo et al. [35] propose a new storage mechanism based on a redundant residual number system to reduce the storage overhead of account information on each node instead of the whole ledger. In the proposed scheme, the information for each account is partially stored on each node as its **residual number (RNS)** related to a module assigned to the node. Since the modules can be an order of magnitude smaller than the normal account information, the storage utilization of the account information on each node can be dramatically reduced.

In UTXO-based blockchain, the increasing size of the UTXO set will soon become the biggest obstacle for nodes to validate a high amount of transactions in time, and will eventually become the bottleneck to implement high-throughput blockchains. Jiang et al. [41] try to minimize the footprint of unspent transactions with a shorter representation form of key and value. For the key, a shorter transaction ID and index is provided by intercepting a part of the original full key, but at the same time, duplicate keys may appear. Once there is a conflict problem, it will query an extra table, which stores the original full key and the corresponding value of these key-value pairs. In addition, a user usually has many UTXOs and the same user information is duplicated in each entry. For the value, a two-level hash mapping technique is proposed for eliminating the redundant data. Specifically, every unique user address is stored as a key in the second table whose value domain is set to null, and the value domain in the first table stores a pointer pointing to use address data. To adapt to the proposed method, some modifications have been made to the underlying database. Compared to the original, the read operation adds steps to access the extra table and the second hash table while the write operation adds steps to compare with possibly conflicting keys, write to the extra table, and access to the second hash table, which adds to the complexity of the database.

The methods of data coding also face an extremely large overhead to recover the entire ledger via some assistance since each node only stores a part of the whole ledger. Therefore, when a single point or range of data is inquired, the steps of restoring the original data are added, which increases the complexity of the system.

3.3.2 Optimization for Query Engine. To improve query efficiency, several techniques are proposed to build an additional external query-preferred database on blockchain systems. The design of these works mainly includes three parts: data monitoring and analyzing module, external database engine, and querying API. The data monitoring and analyzing module monitors the data of a blockchain system and exports the required data to an external query-preferred database engine. Then, a set of API is provided to serve the users' requests. In the following, we will introduce these implementations in detail.

Lesavich and Lesavich [49] propose a method for storage and a system for retrieval of blockchains with Galois fields, respectively. One or more blocks for a blockchain are securely stored and retrieved with a modified Galois Field on a cloud or **peer-to-peer (P2P)** communications network. The modified Galois field provides additional layers for security and privacy for blockchains. The blocks are securely stored and retrieved for cryptocurrency transactions including, but not limited to, BITCOIN transactions and other cryptocurrency transactions.

Li et al. [55] develop EtherQL, an efficient query layer for Ethereum. EtherQL automatically synchronizes new block data in a timely manner and stores them in a dedicated database to ensure efficient and accurate queries. EtherQL provides highly efficient query primitives for analyzing blockchain data, including range queries and top-k queries, which can be flexibly integrated with other applications.

To provide efficient and verifiable data query services for blockchain-based systems, Peng et al. [71] propose a **Verifiable Query Layer (VQL)** middleware. The middleware extracts transactions stored in the underlying blockchain system and reorganizes them in databases to provide

various query services for public users. To prevent falsified data from being stored in the middleware, a cryptographic hash value is calculated for each constructed database. The database fingerprint including the hash value and some database properties will be first verified by miners and then stored in the blockchain.

Hanley and Tewari [39] present a blockchain system that uses off-chain centralized data storage. The system provides instant access to patients' and medical professionals' medical records from anywhere. By assigning each medical record a pseudo-anonymous identifier, a second layer "blockchain" for each user can be created to allow for the rapid collection and data queries. The off-chain pseudo-anonymous data storage allows data to remain unencrypted. Thus, it enables the rapid generation of anonymous medical datasets that can be used for machine learning and data mining, potentially bringing many benefits to the healthcare industry. As the data of the second-tier blockchain system will accumulate all the time, compared with other dedicated databases, the consumption of storage resources is too large and will become more serious with the increment of time.

To analyze a specific historic transaction, all preceding transactions in all preceding blocks need to be run, which includes the execution of smart contracts. Linoy et al. [57] propose an Eidetic blockchain system that supports provenance for both data (blockchain states) and control flow (contracts). Eidetic captures the provenance of the contracts' execution flow (execution flow provenance), their parameters, and the relevant blockchain states before and after each contract call (Why-provenance). This provenance information can then be used to query the execution flow across time in different granularity levels.

In summary, with the help of the query-preferred external databases, these designs make the query process of the blockchain system much more efficient. However, the additional extra database induces a much higher complexity to the implementation of blockchain systems. Moreover, since the storage size of a blockchain system can be extremely large, exporting and reorganizing it with an additional external database takes a much longer time and consumes many resources. As a result, these methods may further increase the data load and decrease the overall performance of blockchain systems.

3.3.3 Optimization for Low Throughput. Public blockchains suffer from low transaction throughput. Two popular public blockchains, Bitcoin and Ethereum, deal with only tens of transactions per second, limiting their deployment [94]. To solve this problem, some research works are proposed to improve the execution speed of transactions by reducing the number of disk I/Os.

Processing a transaction involves executing the transaction and verifying the validation of the execution result. In Ethereum, both execution and verification require reading and writing system states that are stored in a Merkle tree on local storage. Ponnappalli et al. [73] design a custom storage solution, namely, **Distributed, Sharded Merkle Tree (DSM-TREE)**. DSM-TREE is designed in RAINBLOCK which deconstructs miners into three entities: storage nodes that store data, miners that process transactions, and I/O-Helpers that fetch data and proofs from storage nodes and provide them to miners. By having I/O-Helpers prefetch data on behalf of miners, I/O is removed from the critical path of transaction processing; moreover, multiple I/O-Helpers can prefetch data at the same time, which increases the I/O pluralism of the system.

As the blockchain network scales up, the systems face more severe storage performance degradation since inserting and accessing data from the storage layer become more expensive as the total size of the data increases. Chen et al. [15] propose BLock-LSM which co-designs Ethereum's data characteristics and an LSM-tree data structure to optimize the system I/O resource requirements. It introduces a scheme named block number related prefix hashing that makes the insertion of KV entries sequentially to mitigate the write amplification caused by compaction operations during

the data synchronization process. Block-LSM claims that it can increase the synchronize throughput by $3\times$ compared with the original design. However, the way to prefix multiple KV entries undoubtedly sacrifices extra storage space.

3.4 Future Optimization Directions

This subsection lists some future research directions based on the current optimization schemes and existing defects of standard blockchain.

Performance tradeoff between reads and writes. The underlying storage engines employed on blockchain designs mostly use write-friendly databases (such as LevelDB), but efficient reads are also required by real applications. Therefore, the read performance should be carefully considered in further optimization. One possible direction is to improve read performance by exploring the application semantics of blockchain systems. For blockchain systems that adopt KV store as their storage engines, blockchain data are translated into KV items with a unified scheme (e.g., use the hash value of a transaction as the key and the content as the value) and stored indistinguishably.

In the access process of most blockchain systems, there are obvious semantic features. For example, querying a transaction or an account in Ethereum tends to access multiple key-value entries sequentially in the order of metadata to data, whereas these operations to retrieve key-value entries from the underlying storage are regular; that is to say, due to the one-to-one relationship between metadata and data, the next key-value pair to be accessed is confirmed after accessing a key-value pair. In persistent storage, if these data are stored sequentially, the data that is accessed continuously has a strong locality. However, most blockchain systems ignore this semantic characteristic, thus leading to low read performance. Therefore, a future design may take advantage of the spatial locality of a blockchain system (e.g., through designing a prefix-aware hash key) to balance the performance of data reads and writes.

Distributed data query. Due to the increase of data amount, several blockchain systems propose to maintain their data in a distributed approach among nodes. The main problem of this distributed data storage is how to quickly locate which node contains the data required. In the future, we can apply new data structures (such as distributed hash table [63] and learned index [83]) for blockchain systems to speed up the query process by heuristically locating the required data.

Optimization of the underlying storage system. The key-value database has high write performance while relatively low read performance, but in the scenarios of the blockchain system, the workloads usually are read-intensive. For example, in the IOTA blockchain, when issuing a new transaction, the system needs to query all related historical transactions for unique address checking. Since the key-value database is the underlying storage system of the blockchain system, optimizing its internal strategy will not affect the efficiency of the blockchain system itself. Considering the characteristics of the blockchain system, improving the read performance at the underlying database level is also a promising and important optimization direction to increase the overall throughput of blockchain systems.

4 HYBRID BLOCKCHAIN

A Hybrid blockchain system maintains multiple chains in the network, and each node is only allowed to access the data in its specified chain. A hybrid blockchain usually has the same blockchain structure as a standard blockchain. To efficiently maintain multiple chains, one research direction for hybrid blockchain systems is to enable efficient communication between different chains. Moreover, similar to standard blockchain systems, another research direction is to deal with excessive data load. Compared to standard blockchain systems, hybrid blockchain systems may suffer severer excessive data load since the multi-ledger structure increases the amount of data. At the same time, it also leads to increased complexity in data processing and reduces query efficiency. Thus,

Table 4. Overview of Existing Optimization Papers for Hybrid Blockchain

Target problem	Solution	Authors	Year	Weakness
Communication between different blockchains	cross-chain	Wood [95]	2016	No cross-chain transaction
		Kwon and Buchman [46]	2016	Central node exists
Inefficient query mechanism	Metadata model	Gupta et al. [36]	2018	Increase transaction complexity
		Gupta et al. [37]	2018	Increase transaction complexity
	Use other databases	Wai et al. [86]	2019	Increase storage complexity
		Muzammal et al. [65]	2018	Increase storage complexity
Excessive Data Load	data coding	Qi et al. [79]	2020	Data loss
	Data distribution	Zheng et al. [105]	2021	Decrease in decentralization
		Bandara et al. [8]	2021	Decrease in decentralization

next, we will introduce the optimization of hybrid blockchain systems from three aspects, namely, architecture, data structure, and storage engine. Table 4 summarizes these optimizations for the hybrid blockchain system.

4.1 Optimization for Hybrid Blockchain Architecture

Currently, blockchain platform is usually designed for one specific application and communication is not allowed between different platforms. Cross-chain [62] is proposed by changing the hybrid blockchain architecture to enable communication between different chains. The main idea of cross-chain is adding a data structure to the two separate chains so that they can interact with each other through the data structure. Taking digital assets as an example, if the barriers between different chains can be broken, cross-chain transactions of various digital assets can be realized. The two most influential cross-chain technologies are Polkadot [95] and Cosmos [46]. Cosmos targets cross-chain digital asset transactions, while Polkadot focuses on general cross-chain communication.

In Polkadot [95], to support communication, a security mechanism is needed to make mutual trust between each other. Polkadot is the protocol that makes it possible to communicate with other chains and exchange data or assets (tokens) on the basis of maintaining security. Polkadot's backbone network is called the relay chain. It mainly realizes the interconnection with various parachains based on Ethereum. Each secondary chain is a separate blockchain network. Polkadot also aims to upgrade other public chains so that Ethereum can communicate directly with other chains. However, for practical applications, only support communication between different chains may not be enough, which further incurs the design of the cross-chain transaction.

Cosmos [46] is a cross-chain network designed to solve cryptocurrency problems, such as lack of scalability. By using the Cosmos network, users can transfer tokens from one chain to another through internal cross-chain communication. The Cosmos network is composed of various blockchains, which are called zones. Each zone contains a Byzantine fault-tolerant consensus mechanism like Tendermint. The first blockchain in Cosmos will be called the Cosmos hub, which is a multi-asset blockchain used as a central hub. Cosmos realizes the interconnection between different regions through the **inter-blockchain communication (IBC)** protocol running on the backbone network cosmos Hub. In Cosmos, cross-chain transactions need to be implemented through a central hub, which is similar to the existence of a central node. Once it is attached and performs malicious behaviors, the security of the entire blockchain system cannot be guaranteed.

4.2 Optimization for Hybrid Blockchain Data Structure

As mentioned, hybrid blockchain systems have the same data structures as standard blockchain systems. Therefore, the technologies on data structure optimization for standard blockchain systems are also applicable to hybrid blockchain systems. Moreover, several studies [36, 37] are proposed specifically to improve the hybrid blockchain performance.

Gupta et al. [36] discuss the problem of how to efficiently handle temporal queries on Hyperledger Fabric. The temporal nature of the data inserted by the Hyperledger Fabric transactions can be leveraged to support various use cases. They present two models to overcome these limitations and improve the performance of temporal queries on Fabric. The first model creates a copy of each event inserted by a Fabric transaction and stores temporally close events together on Fabric. The second model keeps the event count intact but tags some metadata to each event inserted on Fabric, and temporally close events share the same metadata.

Further, Gupta et al. [37] present a variant based on these two models to better handle skew data patterns. The variant significantly outperforms the approaches presented in [36] when Fabric contains skew data. They also discuss the performance tradeoffs among the variants across various dimensions such as data storage, query performance, event insertion time, and so on. Same as the standard blockchain system, improving query efficiency also increases the complexity of the entire system model, requiring more resources to complete the entire transaction processing.

4.3 Optimization for Hybrid Blockchain Storage Engine

4.3.1 Optimization for Excessive Data Load. Like the standard blockchain, some research papers aim to optimize the excessive data storage problem for hybrid blockchain. They also adopt data sharding and data coding methods. Some up-mentioned methods for the standard blockchain method are still applicable to the hybrid blockchain. Further, there are some other designs for the special application execution environment of the hybrid blockchain.

Qi et al. [79] use erasure coding to reduce the storage pressure of the permissioned blockchain. Specifically, two ideas are designed to achieve scalability and speed up reading performance: (i) a four-phase re-encoding protocol based on **Practical Byzantine Fault Tolerance (PBFT)** to promise the availability of all blocks; and (ii) a multiple replication manner to ensure efficient access of blocks.

Zheng et al. [105] propose Meepo to use multiple execution environments per organization. Meepo also uses the idea of data sharding which includes two main processes: cross-epoch and cross-call. After the consensus of each block ends, each shard begins to communicate across shards. This process is so-called cross-epoch. While cross-call is sharing the message from one shard to another, resulting from cross-shard transactions, such as cross-shard payment. Meepo aggregates cross-calls into several cross-epochs in order and proposes a partial cross-call merging strategy to improve the smart contract flexibility in sharding environments.

Bandara et al. [8] propose Rahasak, in which all blocks, transactions, and asset information are stored in a distributed database. Every blockchain peer comes with a distributed database node; these nodes are connected as a ring cluster. After executing a transaction, state update in a peer is distributed and replicated with other peers via underlying distributed databases' sharding algorithm. However, this approach causes the loss of the immutability of the blockchain itself and further incurs more security problems.

4.3.2 Optimization for Query Engine. To improve the query efficiency of hybrid blockchain systems, Wai et al. [86] and Muzammal et al. [65] propose to optimize the hybrid blockchain storage engine.

Wai et al. [86] first analyze the factors to determine the block size. Based on the analysis, they change the block size to reduce the waiting time of a transaction before creating a block. Therefore, using a proper block size can finally increase the throughput of transactions in data writing. Moreover, the MongoDB database is used as an off-chain to duplicate the metadata of the block. This system can support temporal queries of transactions on a blockchain. The access time of the proposed system can be shortened compared to Hyperledger Fabric.

Table 5. Overview of Existing Optimization Papers for DAG-Based Blockchain

Target problem	Solution	Authors	Year	Weakness
Transaction execution speed	Use NVM	Wang et al. [88]	2019	Not really deployed
	Change transaction verification mode	Ferraro et al. [30]	2018	Reduce transaction security
Low throughput	DAG-based architecture	Li et al. [50]	2018	Unable to resolve malicious transactions
		Li et al. [51]	2020	Unable to resolve malicious transactions
Unique address checking	Cuckoo filter	Shafeeq et al. [82]	2019	Hard to be deployed
	Two-level bloom filter	Wang et al. [90]	2020	Ignores the temporal locality of the addresses
	Hotness aware bloom filter	Zhu et al. [108]	2020	Low safety and fault tolerance

Muzammal et al. [65] implement a protocol named ChainSQL that combines Ripple and distributed databases between all nodes in the Byzantine environment. The protocol can process queries in RDBMS after all transactions are transferred. The transactions are stored in the blockchain, whereas the actual data is stored in the database. Thus, ChainSQL not only provides the instantaneity of the traditional databases but also enhances the security of the blockchain. Similarly, storing metadata in a dedicated database also increases storage complexity and additional resource consumption.

4.4 Future Optimization Directions

For hybrid blockchain systems, the key challenge is how to coordinate the cross-chain data exchange and how to provide efficient queries.

With the development of network topology structure, a hybrid blockchain system may contain hundreds to thousands of sub-chains, thus the verification delays of cross-chain transactions increase exponentially. To alleviate this issue, one potential direction is to combine network sharding technology with cross-chain technology, that is, dividing nodes in the network into different shards. The nodes in each shard only process the transactions within a part of the chains or are initiated by these chains to reduce network load.

On the other hand, the most popular hybrid blockchain transaction technologies are the notary mechanism and chain relay technology, both of which introduce a third party for verification. Therefore, for the query process, a possible improvement direction is to establish a new auxiliary query module in the third party by establishing a SQL-friendly database (e.g., SQLite) to record each transaction and directly retrieve data from the third party when querying.

5 DAG-BASED BLOCKCHAIN

DAG-based blockchains are proposed to provide thousands of TPS. Existing techniques mainly focus on optimizing transaction verification and alleviating the storage burden caused by the unique address checking process. Table 5 summarizes these optimizations for the DAG-based blockchain system.

5.1 Optimization for DAG-Based Blockchain Architecture

Wang et al. [88] present Re-Tangle, a novel DAG-based blockchain acceleration architecture that explores the opportunity of performing massive parallel operations with low energy cost. Specifically, **ReRAM (Resistive Random Access Memory)**, an emerging non-volatile memory with computation capacity, is utilized to optimize the Tangle working process for DAG-based blockchain systems. Re-Tangle consists of a random walking module and a transaction validation module, which transfers Tangle functions into ReRAM-based logic analog computation units. In the random walking module, Re-Tangle maintains an exponentiation translator to reduce its design complexity and improve its computation efficiency for exponentiation calculation. In the transaction validation module, Re-Tangle leverages a highly parallel modular unit to accelerate the validation of different tags in a transaction. Although the use of a ReRAM-based accelerator can significantly improve

overall transaction speed and reduce resource consumption, it has not really landed yet, so currently the solution is practical.

Ferraro et al. [30] propose a simple modification to the attachment mechanism for the Tangle (the IOTA DAG architecture). This modification ensures that all transactions are validated in a finite time and preserves essential features of the popular Monte-Carlo selection algorithm. They propose a new hybrid tip selection algorithm that resolves the dichotomy of double-spend avoidance and orphans and results in a Tangle where all transactions are validated in a finite time. The improved algorithm increases the coming speed of transactions into the chain but reduces the security of the original algorithm to a certain extent, which may lead to the phenomenon of malicious winding into the chain, such as double-spending.

For the standard blockchain, the solution adopted for concurrent block chaining is to select the longest chain. Since the transactions in the short chain are discarded, the overall throughput is reduced. To address this, Conflux [50] organizes blocks into a novel Tree-Graph structure, which is a tree embedded inside a DAG. Conflux assigns a weight to each block, which indicates the amount of finality that the block contributes to its ancestors. To further adjust to environmental changes, Li et al. [51] redesign the consensus protocol that encodes two different block generation strategies: an optimistic strategy that allows fast confirmation and a conservative strategy that ensures the consensus progress. Conflux uses its novel adaptive weight mechanism to combine these two strategies into a unified consensus protocol to cope with different workloads.

5.2 Optimization for Data Storage Engine

Unique address checking is one essential process to guarantee that each transaction from each user account (wallet) is associated with a unique private/public key pair, in which the public and private keys will be utilized as the address and for the signature of the transaction, respectively. Currently, DAG-based blockchain systems usually adopt a database-based approach that relies on the time- and space-consuming database query process to perform the unique address checking process. To address this problem, several approaches have been proposed.

Shafeeq et al. [82] design a cuckoo filter embedded into IOTA lightweight clients to help accelerate this address checking process. This design is based on the assumption that the addresses can be reused after a snapshot, which is performed periodically to reduce Tangle size. They build a Bloom filter for addresses in the current Tangle to promise uniqueness. However, this assumption is unrealistic and makes this design hard to be deployed in practice.

Wang et al. [90] propose a method called ABACUS to utilize a two-level partitioned bloom filter to perform address checking. Partitioned bloom filters are responsible for storing all spent addresses according to their prefixes, and one Sub Bloom Filter of each partitioned bloom filter is kept in memory as a write buffer. However, this approach focuses on optimizing bloom filter update operations so as to speed up unique address checking. In addition, ABACUS assumes all the address checking processes are performed within the paying account and ignores the temporal locality of the addresses.

Zhu et al. [108] propose a **Hotness aware and Fine-grained Bloom Filter (HF-BF)** for unique address checking in DAG-based blockchains. They divide the whole address space into several subspaces according to the address prefix. For each subspace, a Bloom Filter Group that consists of u Bloom Filter units corresponding with K_u hash functions is used to perform the unique address checking. A BFU is a fine-grained bloom filter composed of a bit array for K_u hash functions. To manage the large storage footprint of BFUs and decrease the number of I/Os for checking addresses as much as possible, they design an adaptive BFU management scheme to schedule active BFUs between memory and disk. This solution has certain security risks. Since the address is not stored, once an error occurs in the blockchain network, the transaction cannot be traced back.

Table 6. A Brief Overview of Existing Works of Using Blockchain-Assist Database

Target problem	Authors	Year	Weakness
Data security protection	Ali et al. [3]	2016	Excessive storage consumption
	Grabis et al. [34]	2020	Excessive storage consumption
	Yang et al. [100]	2018	Excessive storage consumption
	Konsta et al. [45]	2021	Excessive storage consumption
	Ali et al. [2]	2019	Excessive storage consumption
	Eltayieb et al. [28]	2020	Excessive storage consumption
	Qin et al. [80]	2021	Excessive storage consumption
	Chen et al. [14]	2017	Excessive storage consumption
Scalability	El-Hindi et al. [27]	2019	Excessive storage consumption
	McConaghy et al. [64]	2016	Inefficient traceability
Secure Query	Li et al. [53]	2018	Inefficient traceability
	Li et al. [52]	2017	Not available for the dynamic data
	Wang et al. [89]	2018	Reduce the scalability
	Do and Ng [24]	2017	Excessive network load
Authentication	Wilkinson et al. [93]	2014	Excessive network load
	Yang et al. [99]	2018	Excessive network load
	Ali et al. [4]	2018	Complex query process

5.3 Future Optimization Directions

In DAG-based blockchain systems, a full node is responsible for making consensus, adding and confirming all transactions, thus consuming lots of computation and memory resources, and making it become the system bottleneck. As the number of transactions increases in a blockchain system, a full node may crash under heavy workloads. Thus, how to mitigate the computation and memory pressure of a full node in a DAG-based blockchain system is an interesting and crucial problem.

Unique address checking is one key process to protect users' security and privacy when transactions are carried out in DAG-based blockchain systems. To guarantee address uniqueness, the address size must be large enough, and with the increment of the number of transactions, it becomes challenging to efficiently perform unique address checking. Several studies propose to adopt bloom filters to speed up the query process, but it still causes many I/O operations. One future optimization direction is to reuse addresses to mitigate resource consumption caused by the unique address checking process. For example, we can use an additional signature scheme (e.g., Merkle OTS signature algorithm) to support reusable addresses based on the current WOTS scheme. Another possible direction is to leverage the snapshot mechanism of DAG-based blockchain systems, that is, unique addresses are only required to be guaranteed in one snapshot period. After each snapshot, addresses with empty balances can be reattached to the DAG structure for address reusing.

6 BLOCKCHAIN-ASSISTED DATABASES

With decentralization, non-tampering, and data consistency, blockchain systems are proposed to assist the design of databases from four aspects: data security protection, scalability, secure query, and authentication. Table 6 provides a summary of these studies.

Data security protection. Benefiting from its safety and immutability, many works choose to use the blockchain as the underlying storage to solve the data security problem. In the blockchain environment, data can be shared safely, and the built-in consensus mechanism ensures that the use of data is subject to certain rules.

Chen et al. [14] propose an improved P2P file database based on IPFS blockchain. They address the low-throughput problem for individual users in IPFS by introducing the role of content service providers. Considering data reliability and availability, storage overhead, and other issues for service providers, they provide a novel zigzag-based storage model to improve the throughput of the block storage model. They chose Bitcoin as the underlying blockchain at the bottom layer of the storage model. All transactions are processed in the upper layer, and then are sent and stored to the blockchain layer. Verified transactions are added to the underlying blockchain, while unverified transactions are discarded. In this environment, the blockchain system is only used as storage. Once the verification service is completed by the node as the service provider, the centralization and non-tampering characteristics of the blockchain system are erased, and the overall security of the system cannot be guaranteed.

El-Hindi et al. [27] utilize blockchains as a storage layer and introduce a database layer on top to extend blockchains by classical data management techniques (e.g., sharding). Further, BlockchainDB provides a standardized key/value-based query interface to facilitate the adoption of blockchains for data sharing. Thus, by using BlockchainDB, we can not only improve the performance and scalability of blockchains for data sharing but also decrease the implementation complexity. On the other hand, the database layer used for expansion will bring additional storage resource consumption.

Ali et al. [3] propose the design and implementation of a new blockchain-based naming and storage system, called Blockstack. Unlike previous blockchain-based systems, Blockstack separates its control and data planes: it keeps only minimal metadata (namely, data hashes and state transitions) in the blockchain and uses external data stores for actual bulk storage. Grabis et al. [34] elaborate a method for efficient distributed storage and sharing of personal data assets within a community of users. The access to these data is controlled using a blockchain and smart contracts that define access conditions. Yang et al. [100] propose the smart toy edge computing-oriented data exchange accounting system based on the blockchain. Through smart contracts, each node in a data-exchange P2P network makes consensus and endorsements for each other, queries the transaction record, and checks the payment bill from its local blockchain peer. The same as [100], there are also some other works [2, 28, 45, 80] that use the storage of data on the blockchain and the use of smart contracts for secure cloud data sharing.

Scalability. Several studies adopt blockchain features to distributed databases for scalability improvement [53, 64]. For example, BigchainDB [64] leverages the blockchain characteristics like decentralized control, immutability, creation, and movement of digital assets on a distributed database. BigchainDB is jointly constructed by database nodes (such as MongoDB instances) available to the enterprise. These nodes store immutable information about assets in a synchronized manner. By implementing Tendermint's BFT consensus algorithm [11], the data stored in the network is propagated and synchronized among all nodes, ensuring the integrity of the network. Li et al. [53] propose a blockchain-based security architecture for distributed cloud storage, where users can divide their files into encrypted data chunks and upload those data chunks randomly onto the P2P network nodes that provide free storage capacity. They customize a genetic algorithm to allocate a file block replica between users and data centers in the distributed cloud storage environment. With the equipment of blockchain features, the confidentiality of distributed storage has been enhanced. However, this method is only suitable for data environments that cannot be changed anymore. At the same time, the file is divided into data blocks, resulting in a stronger data dispersion, making the query more difficult.

Secure query. Several studies utilize blockchains to ensure secure query of encrypted data in databases [24, 52]. Li et al. [52] combine blockchains with **SSE (Searchable Symmetric Encryption)** to store encrypted data in a decentralized manner. To support data retrieval, they

innovatively construct an SSE model using blockchains (SSE-using-BC) to ensure data privacy. They store an encrypted file and its index as a transaction on a blockchain for subsequent queries.

Do and Ng [24] introduce a system that leverages blockchain technology to provide secure data storage with keyword search service. The system allows clients to upload their data in an encrypted form. Moreover, the system can distribute the data content to cloud nodes and ensure data availability using cryptographic techniques. In the system, data owners can grant permission for others to search on the data. Finally, the system supports private keyword search over encrypted datasets.

Wang et al. [89] study the data storage and sharing scheme for decentralized storage systems and propose a framework that combines the decentralized storage system interplanetary file system, the Ethereum blockchain, and **attribute-based encryption (ABE)** technology. Based on the smart contracts of Ethereum, the keyword search function on the ciphertext of the decentralized storage systems is implemented to solve the problem that the cloud server may not return all searched results or return wrong results in the traditional cloud storage systems. It is relatively novel to provide search services based on smart contracts, but at the same time, searching will consume a lot of storage resources and reduce the scalability of the entire system.

Authentication. Several studies have put forward the concept of verifiable databases, in which the main idea is to allow databases to be verified and shared. Several works [4, 93] propose to implement this concept by storing data in traditional databases and storing the digest of the data in an underlying blockchain. Wilkinson et al. [93] devise a new model in which a blockchain system serves as the backbone for a distributed application, Metadisk. This application operates autonomously as a peer-to-peer network of nodes running open-source code. A cryptocurrency will serve as both an incentive and payment mechanism, while a separate blockchain will be used as a datastore for file metadata. Ali et al. [4] design a blockchain-based data storage framework for PingER (a worldwide end-to-end Internet performance measurement project) to remove its total dependence on a centralized repository. They adopt the permissioned blockchain and **Distributed Hash Tables (DHTs)** for this purpose. In the proposed framework, file metadata are stored on the blockchain, whereas file contents are stored off-chain through DHT at multiple locations using a peer-to-peer network with PingER monitoring agents. This design provides decentralized storage, distributed processing, and efficient lookup capabilities to the PingER framework.

Yang et al. [99] propose a novel blockchain-based data deletion scheme, which can make the deletion operation more transparent. The data owner can verify the deletion result no matter how malevolently the cloud server behaves. In addition, with the application of blockchain, the proposed scheme can achieve public verification without any verification from a trusted third party. Unlike [93] and [4], the blockchain system is not used to store metadata, but stores the records of data deletion behavior, which provides sufficient security. However, the problem of excessive network load still exists.

7 RELATED WORK

Previous survey papers for blockchain systems mainly focus on system architecture, security challenges, consensus mechanisms, and applications. However, rare survey work investigates the underlying data management system of blockchain systems.

System architecture. Zheng et al. [107] provide a comprehensive overview of blockchain technology, including blockchain architectures and typical consensus algorithms, and list the technical challenges, the latest developments, and possible future trends. Dai et al. [18] introduce the integration of blockchain and the **Internet of Things (IoT)**, propose related architecture recommendations, and outline the open research directions in this promising field.

Security challenges. Feng et al. [29] analyze the privacy threats in blockchain systems and discuss the existing cryptographic defense mechanisms, namely, anonymity and transaction privacy protection. Joshi et al. [42] conduct a comprehensive investigation of blockchain technology by discussing its structure, different consensus algorithms, and the challenges and opportunities faced by data security and privacy in blockchains.

Consensus mechanisms. Wang et al. [91] conduct in-depth research on the latest consensus protocols and discuss several unresolved problems and future research directions. Nguyen and Kim [68] review the blockchain consensus algorithms and introduce typical applications.

Applications. Lao et al. [47] conduct a systematic survey on the key components of an IoT blockchain and study popular blockchain applications. Duy et al. [26] discuss the impact of blockchain technology, as well as the opportunities and challenges of using it in real-world scenarios. Tasatanattakool and Techapanupreeda [85] investigate the blockchain applications using blockchain technology and the challenges they face. Zhang et al. [104] study the application of blockchain traceability technology in various fields, the decentralized application of blockchains, and the application of other blockchains in data security protection. Al-Jaroodi and Mohamed [1] review different industrial applications that have adopted blockchain technologies, and discuss the opportunities, benefits, and challenges of incorporating blockchains into different industries.

8 CONCLUSION

Blockchain with its key features—decentralization, persistence, and auditability—shows great promise. In this article, we provide a comprehensive overview of the underlying data management techniques of blockchain systems. We first divide the blockchain data management techniques into four layers, namely, blockchain architecture, blockchain data structure, blockchain storage engine, and blockchain-assisted database, for three typical kinds of blockchain systems: standard blockchain, hybrid blockchain, and DAG-based blockchain. We then summarize the existing solutions and ideas that focus on optimizing the blockchain architecture, blockchain data structure, and blockchain storage engine. Finally, we give an overview of existing works of using the blockchain-assisted database. With the continuous emergence of applications based on blockchain technology, we plan to conduct in-depth investigations in the future to improve the operational efficiency and query efficiency of blockchain systems.

REFERENCES

- [1] Jameela Al-Jaroodi and Nader Mohamed. 2019. Blockchain in industries: A survey. *IEEE Access* 7 (2019), 36500–36515.
- [2] Ikram Ali, Mwitende Gervais, Emmanuel Ahene, and Fagen Li. 2019. A blockchain-based certificateless public key signature scheme for vehicle-to-infrastructure communication in VANETs. *J. Syst. Archit.* 99 (2019).
- [3] Muneeb Ali, Jude C. Nelson, Ryan Shea, and Michael J. Freedman. 2016. Blockstack: A global naming and storage system secured by blockchains. In *USENIX Annual Technical Conference (ATC'16)*.
- [4] Saqib Ali, Guojun Wang, Bebo White, and Roger Leslie Cottrell. 2018. A blockchain-based decentralized data storage and access framework for PingER. In *IEEE International Conference on Trust, Security and Privacy in Computing and Communications/IEEE International Conference on Big Data Science and Engineering (TrustCom/BigDataSE'18)*.
- [5] N. Anita and M. Vijayalakshmi. 2019. Blockchain security attack: A brief survey. In *International Conference on Computing, Communication and Networking Technologies (ICCCNT'19)*.
- [6] Frederik Armknecht, Ghassan O. Karame, Avikarsha Mandal, Franck Youssef, and Erik Zenner. 2015. Ripple: Overview and outlook. In *8th International Conference on Trust and Trustworthy Computing (TRUST'15)*.
- [7] Adam Back, Matt Corallo, Luke Dashjr, Mark Friedenbach, Gregory Maxwell, Andrew Miller, Andrew Poelstra, Jorge Timón, and Pieter Wuille. 2014. Enabling blockchain innovations with pegged sidechains. <http://www.opensciencereview.com/papers/123/enablingblockchain-innovations-with-pegged-sidechains>.
- [8] Eranga Bandara, Xueping Liang, Peter Foytik, Sachin Shetty, Nalin Ranasinghe, and Kasun De Zoysa. 2021. Rahasak—scalable blockchain architecture for enterprise applications. *J. Syst. Archit.* 116 (2021), 102061.
- [9] Joseph Bonneau, Andrew Miller, Jeremy Clark, Arvind Narayanan, Joshua A. Kroll, and Edward W. Felten. 2015. SoK: Research perspectives and challenges for bitcoin and cryptocurrencies. In *Symposium on Security and Privacy (SP'15)*.

- [10] J. D. Bruce. 2014. The mini-blockchain scheme. *White Paper*.
- [11] Ethan Buchman, Jae Kwon, and Zarko Milosevic. 2018. The latest gossip on BFT consensus. *CoRR* abs/1807.04938.
- [12] Vitalik Buterin. 2014. A next-generation smart contract and decentralized application platform. *White Paper* 3, 37 (2014).
- [13] Christian Cachin. 2016. Architecture of the hyperledger blockchain fabric. In *Workshop on Distributed Cryptocurrencies and Consensus Ledgers*.
- [14] Yongle Chen, Hui Li, Kejiao Li, and Jiyang Zhang. 2017. An improved P2P file system scheme based on IPFS and Blockchain. In *IEEE International Conference on Big Data (BigData'17)*.
- [15] Zehao Chen, Bingzhe Li, Xiaojun Cai, Zhiping jia, Yi Wang, Zili Shao, and Zhaoyan Shen. 2021. Block-LSM: An ether-aware block-ordered LSM-tree based key-value storage engine. In *International Conference on High Performance Computing and Communications (ICCD'21)*.
- [16] Anton Churymov. 2016. Byteball: A decentralized system for storage and transfer of value. <https://byteball.org/Byteball.pdf>.
- [17] Matt Corallo. 2016. Bip152: Compact block relay. See <https://github.com/bitcoin/bips/blob/master/bip-0152.mediawiki>.
- [18] Hong-Ning Dai, Zibin Zheng, and Yan Zhang. 2019. Blockchain for Internet of Things: A survey. *IEEE Internet Things J.* 6, 5 (2019), 8076–8094.
- [19] Mingjun Dai, Shengli Zhang, Hui Wang, and Shi Jin. 2018. A low storage room requirement framework for distributed ledger in blockchain. *IEEE Access* 6 (2018), 22970–22975.
- [20] Xiaohai Dai, Jiang Xiao, Wenhui Yang, Chaofan Wang, and Hai Jin. 2019. Jidar: A jigsaw-like data reduction approach without trust assumptions for bitcoin system. In *IEEE International Conference on Distributed Computing Systems (ICDCS'19)*.
- [21] Hung Dang, Tien Tuan Anh Dinh, Dumitrel Loghin, Ee-Chien Chang, Qian Lin, and Beng Chin Ooi. 2019. Towards scaling blockchain systems via sharding. In *International Conference on Management of Data (SIGMOD'19)*.
- [22] Donghui Ding, Xin Jiang, Jiaping Wang, Hao Wang, Xiaobing Zhang, and Yi Sun. 2019. Txilm: Lossy block compression with salted short hashing. *CoRR* abs/1906.06500 (2019).
- [23] M. Divya and Nagaveni B. Biradar. 2018. IOTA-next generation block chain. *Int. J. Eng. Comput. Sci. (IJECS)* 7, 4 (2018), 23823–23826.
- [24] Hoang Giang Do and Wee Keong Ng. 2017. Blockchain-based system for secure data storage with private keyword search. In *IEEE World Congress on Services (SERVICES'17)*.
- [25] Panagiotis Drakatos, Erodotos Demetriou, Stavroulla Koumou, Andreas Konstantinidis, and Demetrios Zeinalipour-Yazti. 2021. Towards a blockchain database for massive IoT workloads. In *IEEE International Conference on Data Engineering Workshops (ICDEW'21)*.
- [26] Phan The Duy, Do Thi Thu Hien, Do Hoang Hien, and Van-Hau Pham. 2018. A survey on opportunities and challenges of Blockchain technology adoption for revolutionary innovation. In *Proceedings of the 9th International Symposium on Information and Communication Technology (SolCT'18)*.
- [27] Muhammad El-Hindi, Martin Heyden, Carsten Binnig, Ravi Ramamurthy, Arvind Arasu, and Donald Kossmann. 2019. BlockchainDB—towards a shared database on blockchains. In *International Conference on Management of Data (SIGMOD'19)*.
- [28] Nabeil Eltayieb, Rashad Elhabob, Alzubair Hassan, and Fagen Li. 2020. A blockchain-based attribute-based signcryption scheme to secure data sharing in the cloud. *J. Syst. Archit.* 102 (2020).
- [29] Qi Feng, Debiao He, Sherali Zeadally, Muhammad Khurram Khan, and Neeraj Kumar. 2019. A survey on privacy protection in blockchain system. *J. Netw. Comput. Appl.* 126 (2019), 45–58.
- [30] Pietro Ferraro, Christopher K. King, and Robert Shorten. 2019. IOTA-based directed acyclic graphs without orphans. *CoRR* abs/1901.07302 (2019).
- [31] Junying Gao, Bo Li, and Zhihuai Li. 2018. Blockchain storage analysis and optimization of bitcoin miner node. In *Communications, Signal Processing, and Systems (CSPS'18)*.
- [32] Chunpeng Ge, Xinshu Ma, and Zhe Liu. 2020. A semi-autonomous distributed blockchain-based framework for UAVs system. *J. Syst. Archit.* 107 (2020), 101728.
- [33] Sanjay Ghemawat and Jeff Dean. 2011. LevelDB.
- [34] Janis Grabis, Vlado Stankovski, and Roberts Zarins. 2020. Blockchain enabled distributed storage and sharing of personal data assets. In *IEEE International Conference on Data Engineering Workshops (ICDEW'20)*.
- [35] Zhaohui Guo, Zhen Gao, Haojuan Mei, Ming Zhao, and Jinsheng Yang. 2019. Design and optimization for storage mechanism of the public blockchain based on redundant residual number system. *IEEE Access* 7 (2019), 98546–98554.
- [36] Himanshu Gupta, Sandeep Hans, Kushagra Aggarwal, Sameep Mehta, Bapi Chatterjee, and Praveen Jayachandran. 2018. Efficiently processing temporal queries on hyperledger fabric. In *IEEE International Conference on Data Engineering (ICDE'18)*.

- [37] Himanshu Gupta, Sandeep Hans, Sameep Mehta, and Praveen Jayachandran. 2018. On building efficient temporal indexes on hyperledger fabric. In *IEEE International Conference on Cloud Computing (CLOUD'18)*.
- [38] Siyuan Han, Zihuan Xu, and Lei Chen. 2018. Jupiter: A blockchain platform for Mobile devices. In *IEEE International Conference on Data Engineering (ICDE'18)*.
- [39] Mark Hanley and Hitesh Tewari. 2018. Managing lifetime healthcare data on the blockchain. In *IEEE SmartWorld, Ubiquitous Intelligence & Computing, Advanced & Trusted Computing, Scalable Computing & Communications, Cloud & Big Data Computing, Internet of People and Smart City Innovation (SmartWorld/SCALCOM/UIC/ATC/CBDCOM/IOP/SCI)*.
- [40] intotheblock. 2017. Bitcoin Blockchain Size Over Time. <http://blockchain.info/charts/blocks-size>.
- [41] Shuhao Jiang, Jiajun Li, Shijun Gong, Junchao Yan, Guihai Yan, Yi Sun, and Xiaowei Li. 2020. BZIP: A compact data memory system for UTXO-based blockchains. *J. Syst. Archit.* 109 (2020), 101809.
- [42] Archana Prashanth Joshi, Meng Han, and Yan Wang. 2018. A survey on security and privacy issues of blockchain technology. *Math. Found. Comput.* 1, 2 (2018), 121.
- [43] Yeonsoo Kim, Seongho Jeong, Kamil Jezek, Bernd Burgstaller, and Bernhard Scholz. 2021. An off-the-chain execution environment for scalable testing and profiling of smart contracts. In *USENIX Annual Technical Conference (ATC'21)*.
- [44] Eleftherios Kokoris-Kogias, Philipp Jovanovic, Linus Gasser, Nicolas Gailly, Ewa Syta, and Bryan Ford. 2018. OmniLedger: A secure, scale-out, decentralized ledger via sharding. In *IEEE Symposium on Security and Privacy (SP'18)*.
- [45] Alyzia Konsta, Ioannis Mytilinis, Katerina Doka, Sotiris Niarchos, and Nectarios Koziris. 2021. Clouseau: Blockchain-based data integrity for HDFS clusters. In *IEEE International Conference on Data Engineering (ICDE'21)*.
- [46] Jae Kwon and Ethan Buchman. 2016. Cosmos: A network of distributed ledgers. <https://cosmos.network/whitepaper>.
- [47] Laphou Lao, Zecheng Li, Songlin Hou, Bin Xiao, Songtao Guo, and Yuanyuan Yang. 2020. A survey of IoT applications in blockchain systems: Architecture, consensus, and traffic modeling. *ACM Comput. Surv. (CSUR)* 53, 1 (2020), 1–32.
- [48] Sergio Demian Lerner. 2015. DagCoin: A cryptocurrency without blocks. *White Paper*.
- [49] Stephen Lesavich and Zachary C. Lesavich. 2017. Method and system for storage and retrieval of blockchain blocks using galois fields. U.S. Patent No. 9,569,771.
- [50] Chenxing Li, Peilun Li, Wei Xu, Fan Long, and Andrew Chi-Chih Yao. 2018. Scaling Nakamoto consensus to thousands of transactions per second. *CoRR* abs/1805.03870 (2018).
- [51] Chenxing Li, Peilun Li, Dong Zhou, Zhe Yang, Ming Wu, Guang Yang, Wei Xu, Fan Long, and Andrew Chi-Chih Yao. 2020. A decentralized blockchain with high throughput and fast confirmation. In *USENIX Annual Technical Conference (ATC'20)*.
- [52] Huijie Li, Fangguo Zhang, Jiejie He, and Haibo Tian. 2017. A searchable symmetric encryption scheme using BlockChain. *CoRR* abs/1711.01030 (2017).
- [53] Jiaxing Li, Jigang Wu, and Long Chen. 2018. Block-secure: Blockchain based scheme for secure P2P cloud storage. *Inf. Sci.* 465 (2018), 219–231.
- [54] Xiaoqi Li, Peng Jiang, Ting Chen, Xiapu Luo, and Qiaoyan Wen. 2020. A survey on the security of blockchain systems. *Future Gener. Comput. Syst.* 107 (2020), 841–853.
- [55] Yang Li, Kai Zheng, Ying Yan, Qi Liu, and Xiaofang Zhou. 2017. EtherQL: A query layer for blockchain system. In *1055 International Conference on Database Systems for Advanced Applications (DASFAA'17)*.
- [56] Iuon-Chang Lin and Tzu-Chun Liao. 2017. A survey of blockchain security issues and challenges. *Int. J. Netw. Secur.* 19, 5 (2017), 653–659.
- [57] Shlomi Linoy, Suprio Ray, and Natalia Stakhanova. 2020. Towards eidetic blockchain systems with enhanced provenance. In *IEEE International Conference on Data Engineering Workshops (ICDE'20)*.
- [58] Jing Liu and Zhentian Liu. 2019. A survey on security verification of blockchain smart contracts. *IEEE Access* 7 (2019), 77894–77904.
- [59] Yinqiu Liu, Kun Wang, Yun Lin, and Wenyao Xu. 2019. *LightChain*: A lightweight blockchain system for industrial Internet of Things. *IEEE Trans. Ind. Inform.* 15, 6 (2019), 3571–3581.
- [60] Eric Lombrozo, Johnson Lau, and Pieter Wuille. 2015. *Segregated Witness (Consensus Layer)*. Bitcoin Core Development Team, Technical Report BIP 141.
- [61] Loi Luu, Viswesh Narayanan, Chaodong Zheng, Kunal Baweja, Seth Gilbert, and Prateek Saxena. 2016. A secure sharding protocol for open blockchains. In *Computer and Communications Security (SIGSAC'16)*.
- [62] Léonard Lys, Arthur Micoulet, and Maria Potop-Butucaru. 2020. Atomic cross chain swaps via relays and adapters. In *CryBlock@MOBICOM 2020: Proceedings of the 3rd Workshop on Cryptocurrencies and Blockchains for Distributed Systems*.
- [63] Ward Douglas Maurer and Theodore Gyle Lewis. 1975. Hash table methods. *ACM Comput. Surv. (CSUR)* 7, 1 (1975), 5–19.

- [64] Trent McConaghy, Rodolphe Marques, Andreas Müller, Dimitri De Jonghe, Troy McConaghy, Greg McMullen, Ryan Henderson, Sylvain Bellemare, and Alberto Granzotto. 2016. Bigchaindb: A scalable blockchain database. *White Paper, BigChainDB* (2016).
- [65] Muhammad Muzammal, Qiang Qu, Bulat Nasrulin, and Anders Skovsgaard. 2018. A blockchain database application platform. *CoRR* abs/1808.05199 (2018).
- [66] Satoshi Nakamoto. 2019. *Bitcoin: A Peer-to-peer Electronic Cash System*. Technical Report. Manubot.
- [67] Raiden Network. 2018. What is the Raiden Network. <https://raiden.network>.
- [68] Giang-Truong Nguyen and Kyungbaek Kim. 2018. A survey about consensus algorithms used in blockchain. *J. Inf. Process. Syst.* 14, 1 (2018), 101–128.
- [69] Asutosh Palai, Meet Vora, and Aashaka Shah. 2018. Empowering light nodes in blockchains with block summarization. In *International Conference on New Technologies, Mobility and Security (NTMS'18)*.
- [70] Christos Patsonakis and Mema Roussopoulos. 2019. An alternative paradigm for developing and pricing storage on smart contract platforms. In *IEEE International Conference on Decentralized Applications and Infrastructures (DAPP-CON'19)*.
- [71] Zhe Peng, Haotian Wu, Bin Xiao, and Songtao Guo. 2019. VQL: Providing query efficiency and data authenticity in blockchain systems. In *IEEE International Conference on Data Engineering Workshops (ICDE'19)*.
- [72] Doriane Perard, Jérôme Lacan, Yann Bachy, and Jonathan Detchart. 2018. Erasure code-based low storage blockchain node. In *International Conference on Internet of Things (iThings) and Green Computing and Communications (Green-Com) and Cyber, Physical and Social Computing (CPSCom) and Smart Data (SmartData)*.
- [73] Soujanya Ponnappalli, Aashaka Shah, Souvik Banerjee, Dahlia Malkhi, Amy Tai, Vijay Chidambaram, and Michael Wei. 2021. RainBlock: Faster transaction processing in public blockchains. In *USENIX Annual Technical Conference (ATC'21)*.
- [74] Soujanya Ponnappalli, Aashaka Shah, Amy Tai, Souvik Banerjee, Vijay Chidambaram, Dahlia Malkhi, and Michael Wei. 2019. Scalable and efficient data authentication for decentralized systems. *CoRR* abs/1909.11590 (2019).
- [75] Joseph Poon and Vitalik Buterin. 2017. Plasma: Scalable autonomous smart contracts. *White Paper* (2017), 1–47.
- [76] Joseph Poon and Thaddeus Dryja. 2016. The Bitcoin Lightning Network: Scalable Off-chain Instant Payments.
- [77] Serguei Popov. 2018. The tangle. White paper. <http://www.descryptions.com/Iota.pdf>.
- [78] Serguei Popov, Olivia Saa, and Paulo Finardi. 2019. Equilibria in the tangle. *Comput. Indust. Eng.* 136 (2019), 160–172.
- [79] Xiaodong Qi, Zhao Zhang, Cheqing Jin, and Aoying Zhou. 2020. BFT-store: Storage partition for permissioned blockchain via erasure coding. In *IEEE International Conference on Data Engineering (ICDE'20)*.
- [80] Xuanmei Qin, Yongfeng Huang, Zhen Yang, and Xing Li. 2021. A blockchain-based access control scheme with multiple attribute authorities for secure cloud data sharing. *J. Syst. Archit.* 112 (2021), 101854.
- [81] Emanuel Regnath and Sebastian Steinhorst. 2018. LeapChain: Efficient blockchain verification for embedded IoT. In *International Conference on Computer-Aided Design (ICCAD'18)*.
- [82] Sehrish Shafeeq, Sherali Zeadally, Masoom Alam, and Abid Khan. 2020. Curbing address reuse in the IOTA distributed ledger: A cuckoo-filter-based approach. *Trans. Eng. Manage.* 67, 4 (2020), 1244–1255.
- [83] Barbara A. Solomon and Richard M. Felder. 2005. Index of learning styles questionnaire. <http://www.engr.ncsu.edu/learningstyles/ilsweb.html>.
- [84] Yuechen Tao, Bo Li, Jingjie Jiang, Hok Chu Ng, Cong Wang, and Baochun Li. 2020. On sharding open blockchains with smart contracts. In *IEEE International Conference on Data Engineering (ICDE'20)*.
- [85] Pinyaphat Tasatanattakool and Chian Techapanupreeda. 2018. Blockchain: Challenges and applications. In *International Conference on Information Networking (ICOIN'18)*.
- [86] Khin Su Su Wai, Ei Chaw Htoon, and Nwe Nwe Myint Thein. 2019. Storage structure of student record based on hyperledger fabric blockchain. In *International Conference on Advanced Information Technologies (ICAIT'19)*.
- [87] Jiaping Wang and Hao Wang. 2019. Monoxide: Scale out blockchains with asynchronous consensus zones. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI'19)*.
- [88] Qian Wang, Tianyu Wang, Zhaoyan Shen, Zhiping Jia, Mengying Zhao, and Zili Shao. 2019. Re-tangle: A ReRAM-based processing-in-memory architecture for transaction-based blockchain. In *Proceedings of the International Conference on Computer-Aided Design (ICCAD'19)*.
- [89] Shangping Wang, Yinglong Zhang, and Yaling Zhang. 2018. A blockchain-based framework for data sharing with fine-grained access control in decentralized storage systems. *IEEE Access* 6 (2018), 38437–38450.
- [90] Tianyu Wang, Wenbin Zhu, Qun Ma, Zhaoyan Shen, and Zili Shao. 2020. ABACUS: Address-partitioned bloom filter on address checking for uniqueness in IoT blockchain. In *International Conference on Computer Aided Design (ICCAD'20)*.
- [91] Wenbo Wang, Dinh Thai Hoang, Peizhao Hu, Zehui Xiong, Dusit Niyato, Ping Wang, Yonggang Wen, and Dong In Kim. 2019. A survey on consensus mechanisms and mining strategy management in blockchain networks. *IEEE Access* 7 (2019), 22328–22370.

- [92] Bitcoin Wiki. 2015. Scalability. *Bitcoin Wiki*.
- [93] Shawn Wilkinson, Jim Lowry, and Tome Boshevski. 2014. Metadisk a blockchain-based decentralized file storage application. Technical Report. 1–11.
- [94] J. I. Wong. 2017. Cryptokitties is causing ethereum network congestion. <https://qz.com/1145833/cryptokitties-is-causing-ethereum-network-congestion>.
- [95] Gavin Wood. 2016. Polkadot: Vision for a heterogeneous multi-chain framework. *White Paper*.
- [96] Cheng Xu, Ce Zhang, and Jianliang Xu. 2019. vChain: Enabling verifiable boolean range queries over blockchain databases. In *International Conference on Management of Data (SIGMOD'19)*.
- [97] Yibin Xu. 2018. Section-blockchain: A storage reduced blockchain protocol, the foundation of an autotrophic decentralized storage architecture. In *International Conference on Engineering of Complex Computer Systems (ICECCS'18)*.
- [98] Zihuan Xu, Siyuan Han, and Lei Chen. 2018. CUB, a consensus unit-based storage scheme for blockchain system. In *IEEE International Conference on Data Engineering (ICDE'18)*.
- [99] Changsong Yang, Xiaofeng Chen, and Yang Xiang. 2018. Blockchain-based publicly verifiable data deletion scheme for cloud storage. *J. Netw. Comput. Appl.* 103 (2018), 185–193.
- [100] Jian Yang, Zhihui Lu, and Jie Wu. 2018. Smart-toy-edge-computing-oriented data exchange based on blockchain. *J. Syst. Archit.* 87 (2018), 36–48.
- [101] Mahdi Zamani, Mahnush Movahedi, and Mariana Raykova. 2018. RapidChain: Scaling blockchain via full sharding. In *ACM SIGSAC Conference on Computer and Communications Security (CCS'18)*.
- [102] Ce Zhang, Cheng Xu, Haixin Wang, Jianliang Xu, and Byron Choi. 2021. Authenticated keyword search in scalable hybrid-storage blockchains. In *IEEE International Conference on Data Engineering (ICDE'21)*.
- [103] Ce Zhang, Cheng Xu, Jianliang Xu, Yuzhe Tang, and Byron Choi. 2019. GEM'2-Tree: A gas-efficient structure for authenticated range queries in blockchain. In *IEEE International Conference on Data Engineering (ICDE'19)*.
- [104] Jingyu Zhang, Siqi Zhong, Tian Wang, Han-Chieh Chao, and Jin Wang. 2020. Blockchain-based systems and applications: A survey. *J. Internet Technol.* 21, 1 (2020), 1–14.
- [105] Peilin Zheng, Quanqing Xu, Zibin Zheng, Zhiyuan Zhou, Ying Yan, and Hui Zhang. 2021. Meepo: Sharded consortium blockchain. In *IEEE International Conference on Data Engineering (ICDE'21)*.
- [106] Qiuhong Zheng, Yi Li, Ping Chen, and Xinghua Dong. 2018. An innovative IPFS-based storage model for blockchain. In *International Conference on Web Intelligence (WI'18)*.
- [107] Zibin Zheng, Shaoan Xie, Hongning Dai, Xiangping Chen, and Huaimin Wang. 2017. An overview of blockchain technology: Architecture, consensus, and future trends. In *International Congress on Big Data (BigData Congress'17)*.
- [108] Wenbin Zhu, Qun Ma, Zhaoyan Shen, Tianyu Wang, and Zhiping jia. 2020. HF-BF: A hotness-aware fine-grained bloom filter for unique address checking in IoT blockchain. In *International Conference on High Performance Computing and Communications (HPCC'20)*.

Received January 2021; revised September 2021; accepted November 2021